DAY 4 - BUILDING DYNAMIC FRONTEND COMPONENTS

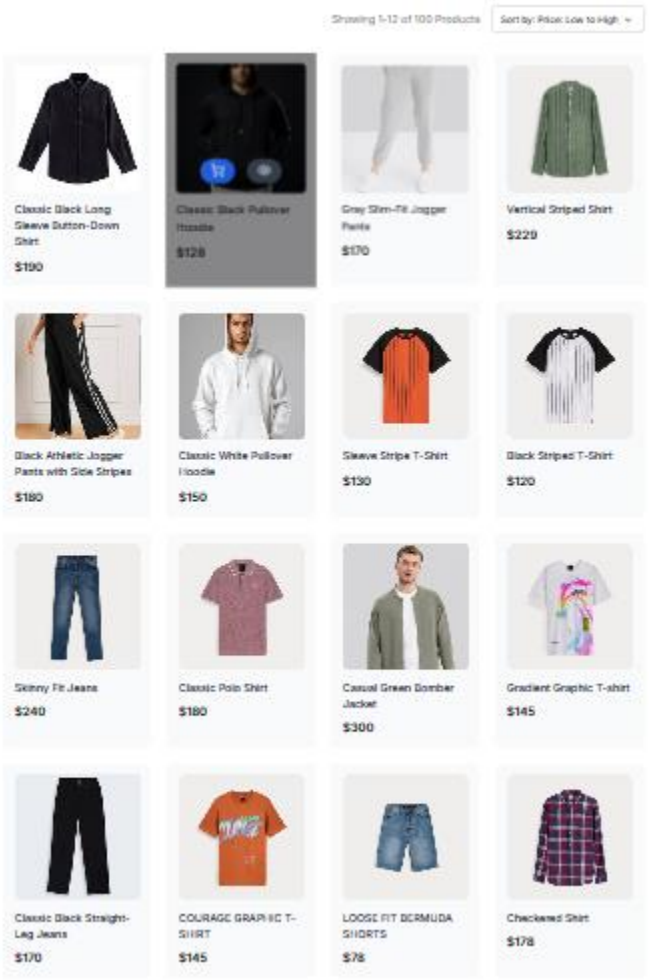
(SHOP.CO)

Day 4 Activity Summary:

On Day 4, we focused on building dynamic frontend components using Sanity CMS and APIs, emphasizing reusable and modular design. State management techniques were implemented for efficient data handling. We also practiced responsive design and UX/UI best practices, preparing for real-world client projects with professional workflows.

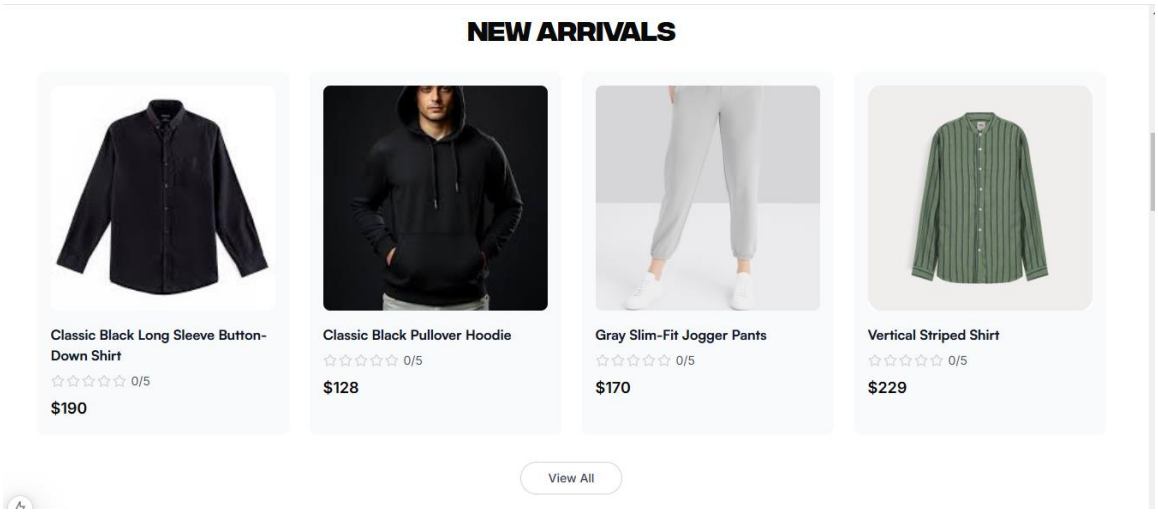
Product Listing:

I fetched product data for the listing using a sanity query and displayed it on the UI. By using Groq queries, I was able to pull accurate and relevant product information to show in a clear and organized way on the website.



New Arrivals:

For the "New Arrivals" section, I used a Groq query to fetch the latest product data from Sanity. This allowed me to display newly added products on the UI, showcasing fresh arrivals with their images, descriptions, and prices in a user-friendly format.



here's a code:

```
app > product > [id] > page.tsx > client
'use client';

import React, { useEffect, useState } from 'react';
import { useParams } from 'next/navigation';
import { useCart } from '@/context/CartContext';
import { toast } from 'react-hot-toast';
import sanityClient from '@sanity/client';
import Testimonials from '@/components/home/testimonials';
import NewArrivals from '@/components/home/top-selling';


const client = sanityClient({
  projectId: 'vc3rco0c',
  dataset: 'production',
  useCdn: true,
});

export default function ProductDetailPage() {
  const { id } = useParams();
  const { dispatch } = useCart();
  const [product, setProduct] = useState<any>(null);
  const [loading, setLoading] = useState(true);
  const [size, setSize] = useState<string>(''); // Selected size
  const [color, setColor] = useState<string>(''); // Selected color
  const [quantity, setQuantity] = useState<number>(1); // Selected quantity
  const [selectedImage, setSelectedImage] = useState<string>(''); // Selected image

  // Fetch product details
  useEffect(() => {
    const fetchProduct = async () => {
      try {
        const fetchedProduct = await client.fetch(
          `*[_id == $id][0]{
```

Product Details and Dynamic Routing:

I fetched product details for the listing using a sanity query with Groq. The data was then displayed on the UI, including product images, descriptions, prices, and key features, ensuring the information was accurate and well-organized for the users.

Product 1:

SHOP.CO Shop On Sale New Arrivals Brands Search for products...

Vertical Striped Shirt

This product is a stylish vertical striped shirt, featuring alternating green and darker green stripes for a sophisticated, yet casual look. The shirt has a simple button-down design with a modern band collar, offering a clean and sharp appearance. Its long sleeves and breathable fabric make it an ideal choice for various occasions, from a laid-back day out to semi-casual gatherings. The subtle stripe pattern adds a unique touch while maintaining versatility, allowing it to pair well with both jeans and chinos.

☆☆☆

$229

Size

Small

Color

Red

Quantity: - 1 +

Add to Cart

Product 2:

Classic Black Pullover Hoodie

This versatile and stylish black hoodie is a must-have in any wardrobe. Featuring a comfortable, soft fabric and a relaxed fit, it is perfect for casual outings or layering for colder weather. The hoodie includes an adjustable drawstring and a spacious front pocket, making it both practical and fashionable. The black color adds a timeless touch, making it easy to pair with jeans, shorts, or joggers for an effortlessly cool look.

☆☆☆

$128

Size

Small

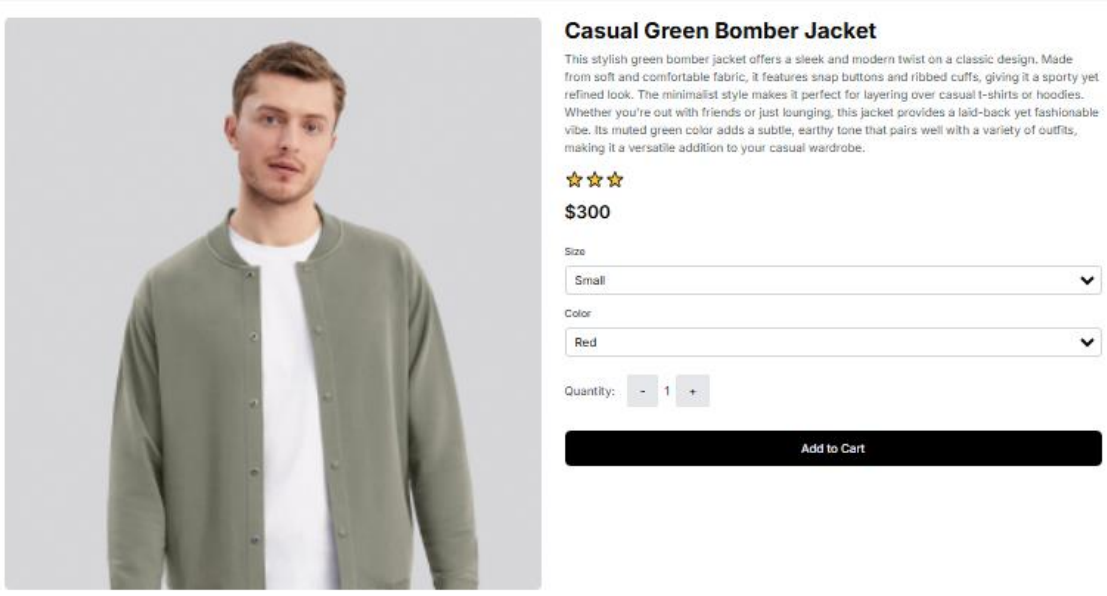
Color

Red

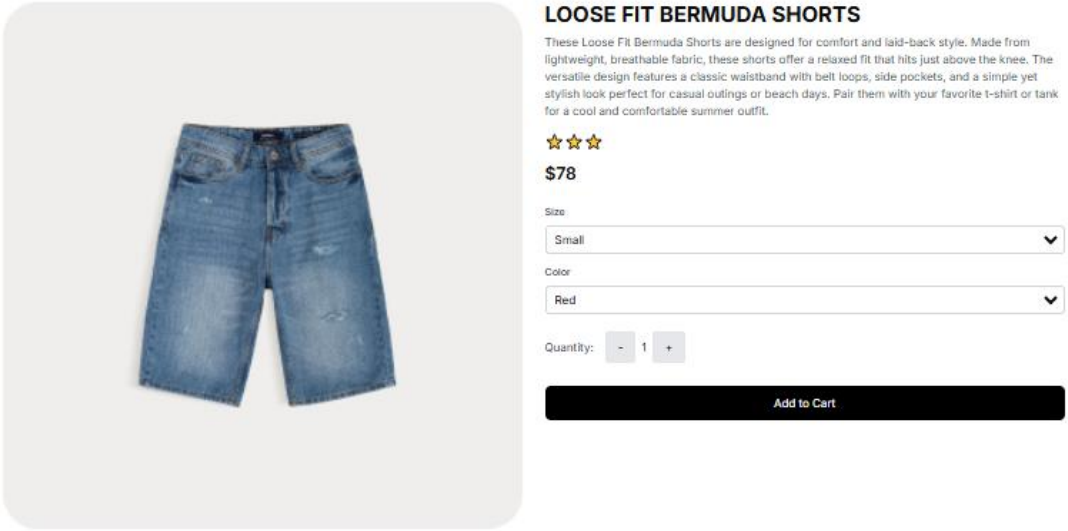
Quantity: - 1 +

Add to Cart

Product 3:

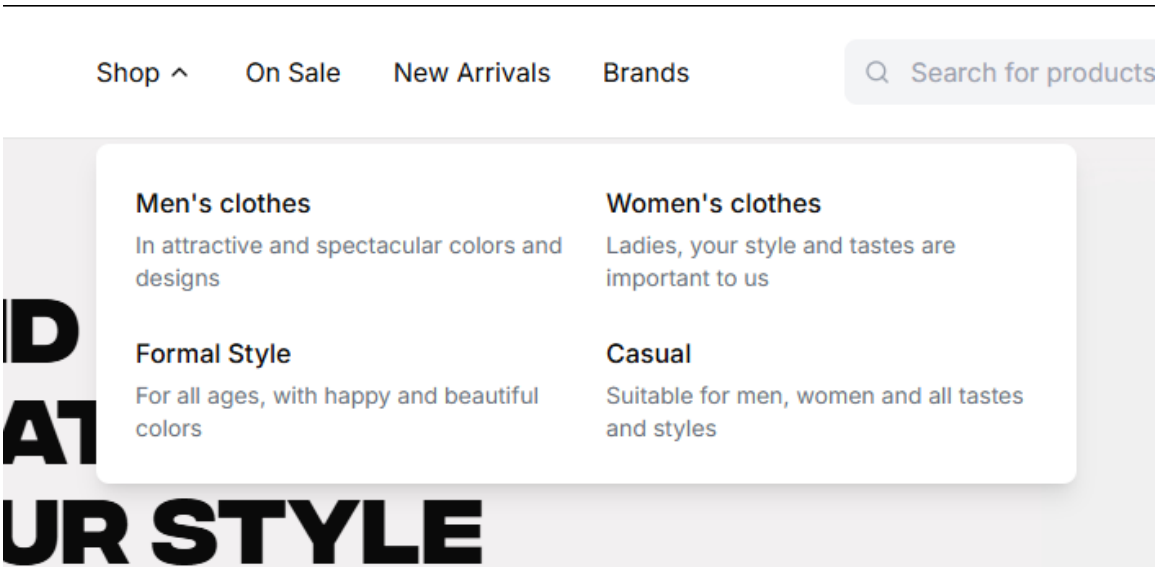


Product 4:

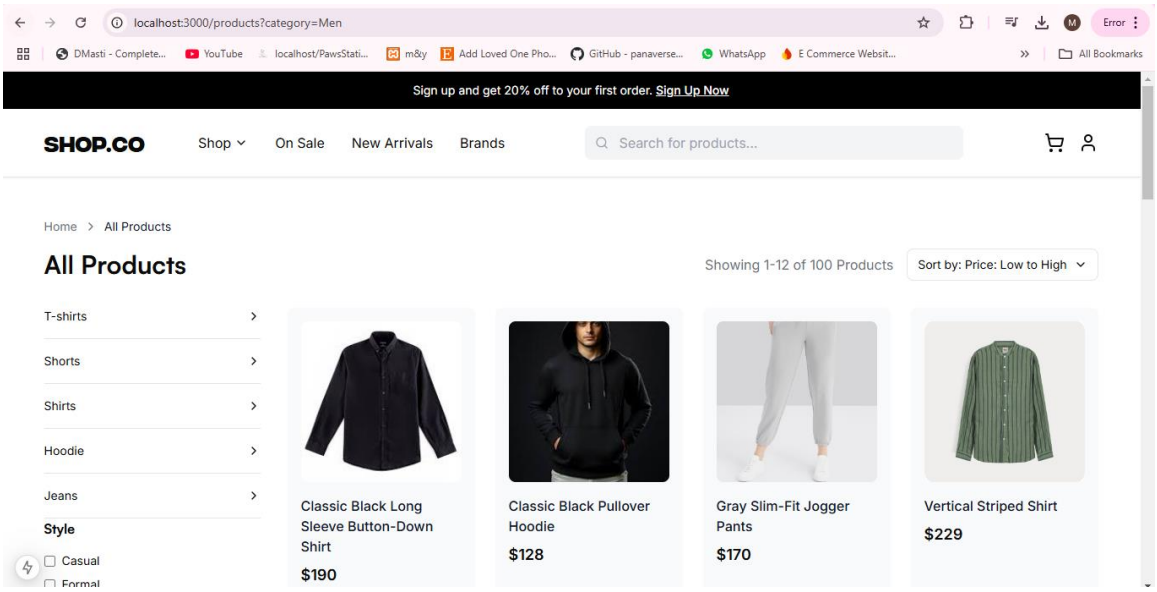


Categories:

For the category components, I fetched product data from Sanity using Groq queries to display items based on their categories. This allowed me to organize products into specific sections like clothing, shoes, and accessories on the UI, making it easier for users to browse and find what they're looking for.

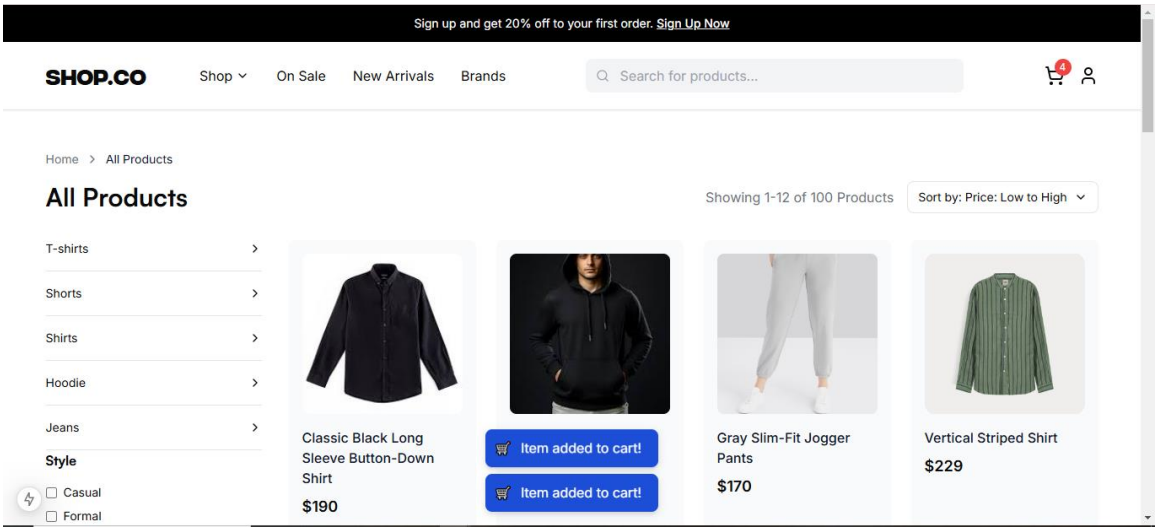



```
export const menuItems = [
  {
    title: "Men's clothes",
    subtitle: "In attractive and spectacular colors and designs",
    href: "/products?category=Men"
  },
  {
    title: "Women's clothes",
    subtitle: "Ladies, your style and tastes are important to us",
    href: "/products?category=Women"
  },
  {
    title: "Formal Style",
    subtitle: "For all ages, with happy and beautiful colors",
    href: "/products?style=formal"
  },
  {
    title: "Casual",
    subtitle: "Suitable for men, women and all tastes and styles",
    href: "/products?style=casual"
  }
]
```



Cart Component:

For the cart components, I implemented a system that fetches and manages cart data, allowing users to add and remove products. Using Groq queries and the Sanity database, I ensured the cart content was accurately displayed on the UI, with product details like name, price, and quantity, and updated dynamically as users interacted with it.

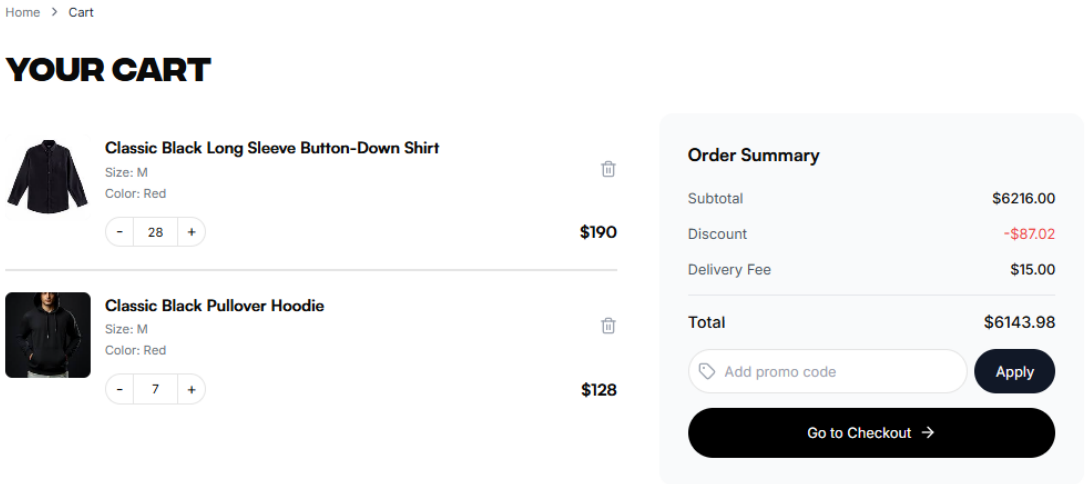


CODE:

```
'use client';

import { ShoppingCart, Eye } from 'lucide-react';
import { useState, useEffect } from 'react';
import { useCart } from '@/context/CartContext';
import { CartItem } from '@/types/cart';
import { toast } from 'react-hot-toast';
import sanityClient from '@sanity/client';
import Link from 'next/link';

const client = sanityClient({
  projectId: "vc3rco0c", // Replace with your project ID
  dataset: 'production', // Replace with your dataset name
  useCdn: true, // Use CDN for faster queries
});

interface Product {
  _id: string;
  name: string;
  price: number;
  originalPrice?: number;
  rating: number;
  image: {
    asset: {
      url: string;
    };
  };
}

export default function ProductCard({ product }: { product: Product }) {
  const { dispatch, state } = useCart();
  const [isClient, setIsClient] = useState(false);
  const [loading, setLoading] = useState(false);
```

Order Summary:

For the order summary, I fetched data from the cart and order details using Groq queries from Sanity. The summary displayed key information such as the total number of items, individual product prices, taxes, shipping costs, and the final total, all updated in real-time as users proceed through the checkout process.

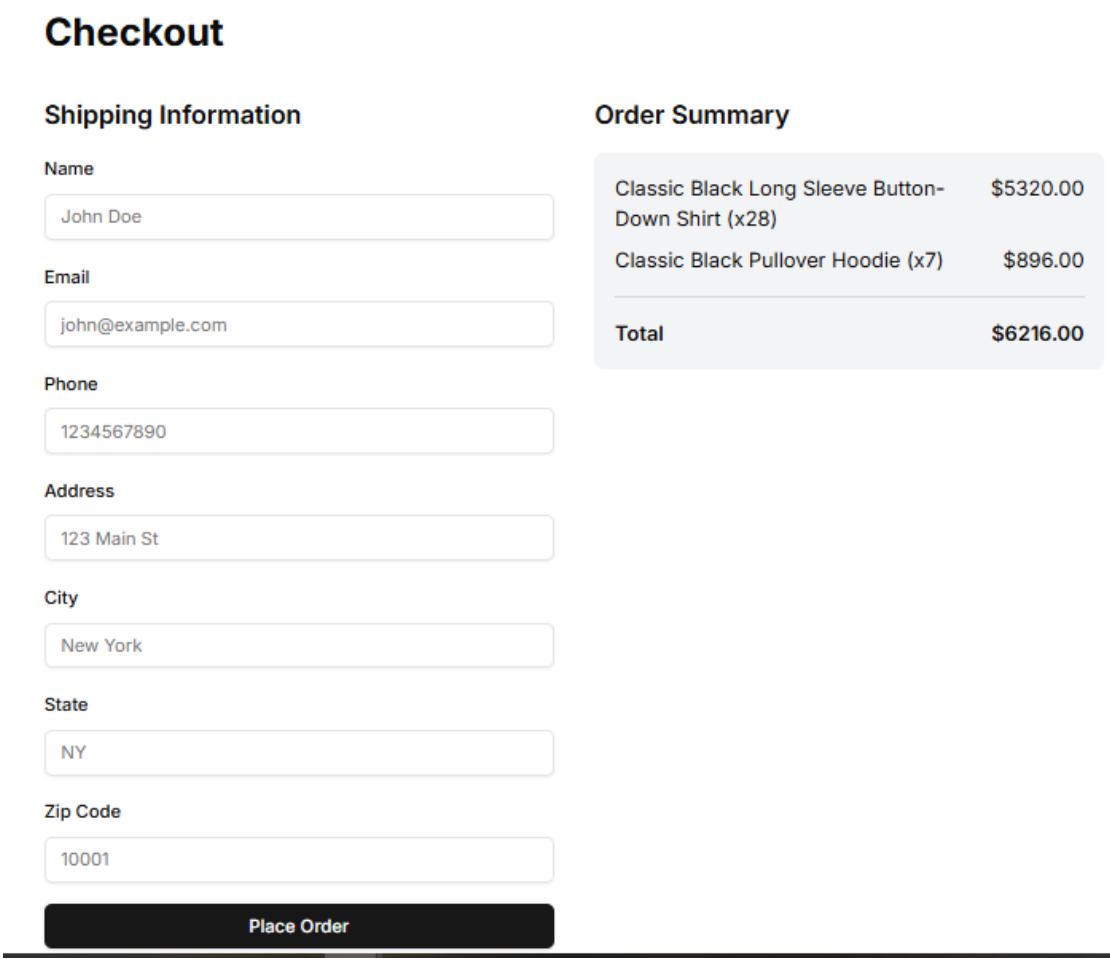


CODE:

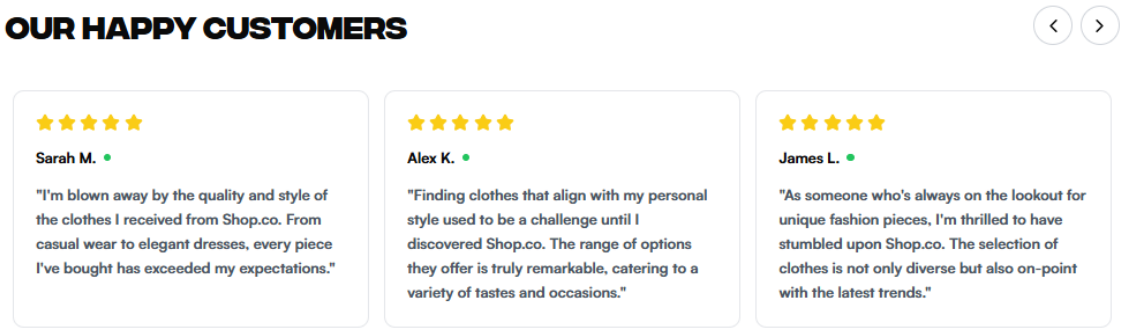
```tsx
interface OrderSummaryProps {
  subtotal: number
  discount: number
  deliveryFee: number
  total: number
}

export function OrderSummary({ subtotal, discount, deliveryFee, total }: OrderSummaryProps) {
  const formatPrice = (price: number) => price.toFixed(2)

  return (
    <div className="bg-gray-50 rounded-2xl p-6 lg:p-8">
      <h2 className={cn("text-xl font-medium mb-6", satoshi.className)}>Order Summary</h2>
      <div className="space-y-4">
        <div className="flex justify-between">
          <span className="text-gray-600">Subtotal</span>
          <span className="font-medium">${formatPrice(subtotal)}</span>
        </div>
        <div className="flex justify-between">
          <span className="text-gray-600">Discount</span>
          <span className="text-red-500">-${formatPrice(discount)}</span>
        </div>
        <div className="flex justify-between">
          <span className="text-gray-600">Delivery Fee</span>
          <span className="font-medium">${formatPrice(deliveryFee)}</span>
        </div>
        <div className="h-px bg-gray-200 my-4" />
        <div className="flex justify-between text-lg font-medium">
          <span>Total</span>
          <span>${formatPrice(total)}</span>
        </div>
```

Checkout Component:

For the checkout component, I integrated the product data from the cart and user information using Groq queries from Sanity. This allowed users to review their order details, enter shipping and payment information, and finalize their purchase. The checkout process was designed to be seamless, with real-time updates on pricing, discounts, and shipping options.



CUSTOMER FEEDBACK



FILTER:

Home > All Products

All Products

T-shirts >

Shorts >

Shirts >

Hoodie >

Jeans >

Style

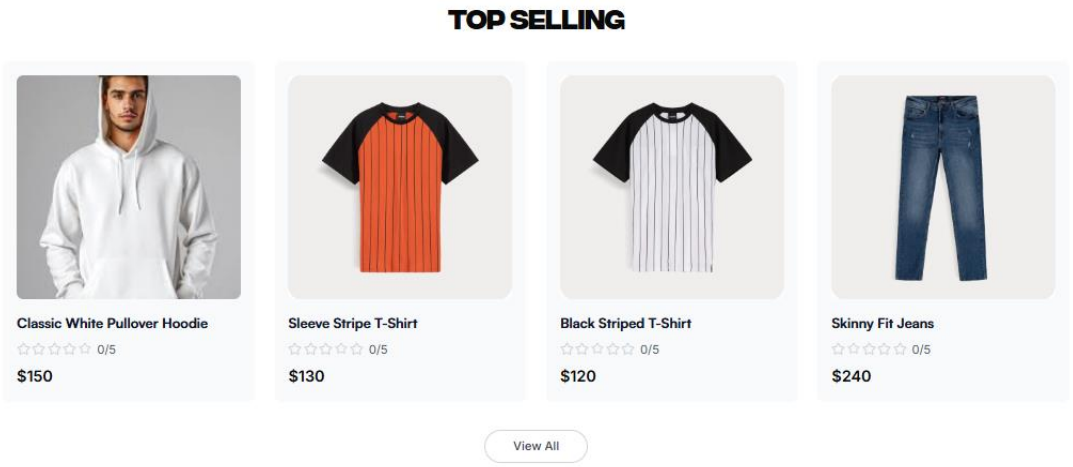
- [ ] Casual
- [ ] Formal
- [ ] Party
- [ ] Gym

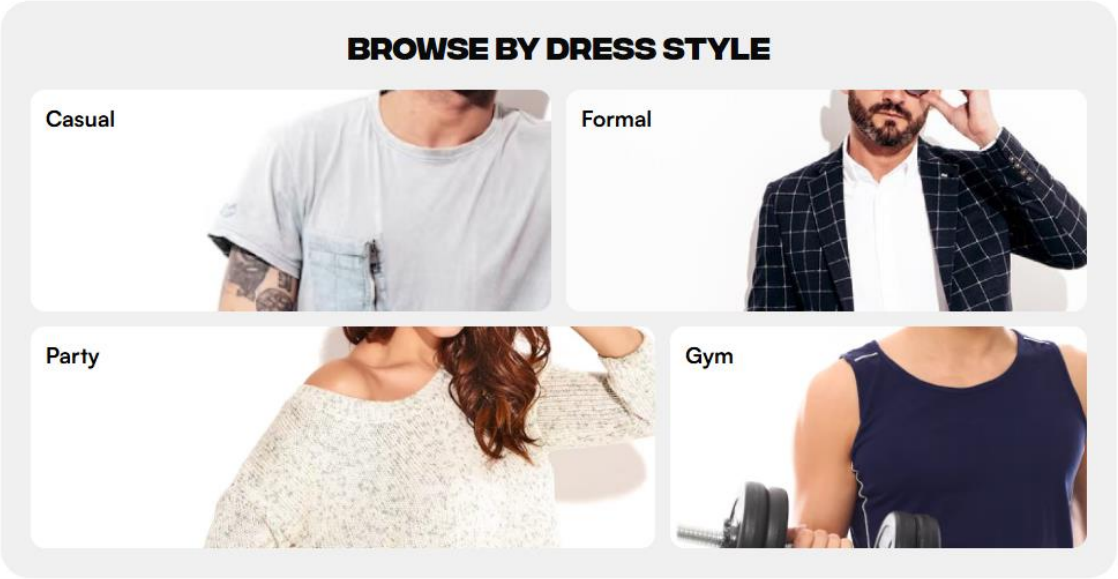
Price

0 1000

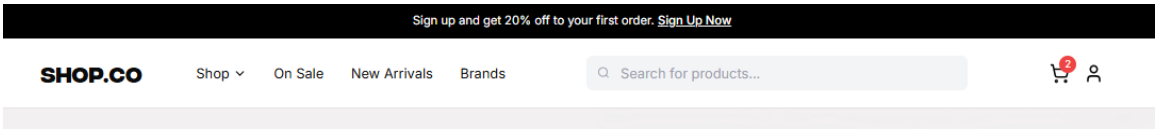
Apply Price Filter

RELATED PRODUCTS:

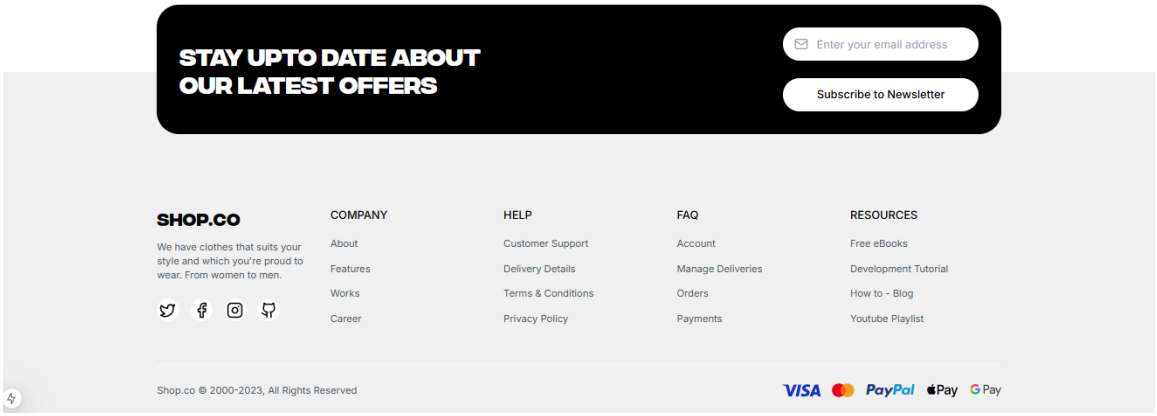


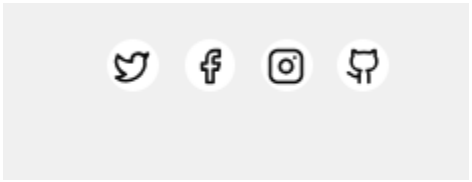


NAVBAR:



FOOTER:





SOCIAL MEDIA:

SCRIPTS CODE:

```
scripts > importData.mjs > ...
1  import { createClient } from '@sanity/client';
2
3  const client = createClient({
4    projectId: "vc3rco0c",
5    dataset: "production",
6    useCdn: true,
7    apiVersion: '2025-01-13',
8    token: "sk6dstRbP4HjcUXlYvBrbteEKQTCtXgp7Nk87TreSP8i8ghdivIDoVG3yXAfRtBxa1XTO0cNkj6NZDZYe8VpUkw3K9C59MGi4FpfuvjZjrfpr2ftLnpodvVZW9MuHdGSMg159AZ
9  });
10
11 async function uploadImageToSanity(imageUrl) {
12   try {
13     console.log(`Uploading image: ${imageUrl}`);
14
15     const response = await fetch(imageUrl);
16     if (!response.ok) {
17       throw new Error(`Failed to fetch image: ${imageUrl}`);
18     }
19
20     const buffer = await response.arrayBuffer();
21     const bufferImage = Buffer.from(buffer);
22
23     const asset = await client.assets.upload('image', bufferImage, {
24       filename: imageUrl.split('/').pop(),
25     });
26
27     console.log(`Image uploaded successfully: ${asset._id}`);
28     return asset._id;
29   } catch (error) {
30     console.error('Failed to upload image:', imageUrl, error);
31     return null;
32   }
33 }
```